

Matricola: _____

Nome: _____

Cognome: _____

ESAME Programmazione Avanzata e Parallela

3 giugno 2026

L'esame consiste in due parti:

- La **prima parte** consiste in 10 domande a risposta multipla sugli argomenti del corso. Ogni domanda può ricevere un punteggio massimo di *due* punti. Affinché una risposta sia considerata valida la scelta *deve essere motivata*. Una risposta errata o non motivata riceverà *zero* punti. *Il punteggio minimo è 12 punti su 20.*
- La **seconda parte** consiste in due esercizi di programmazione. Ogni esercizio può avere un punteggio massimo di *cinque* punti. *Il punteggio minimo è 6 punti su 10.*

IMPORTANTE: *Per raggiungere la sufficienza è necessario raggiungere il punteggio minimo in entrambe le parti.*

Parte 1

Domanda 1

Si supponga di avere il seguente makefile:

```
all: main
```

```
main: A.o B.o C.o  
      gcc $^ -o main
```

```
%.o: %.c  
      gcc -c $< -o $@
```

Se, dopo aver precedentemente invocato `make` viene modificato il file `A.c` e cancellato il file `B.o`, alla successiva invocazione di `make` quali comandi saranno eseguiti

```
gcc -c A.c -o A.o
☐ gcc -c C.c -o C.o
gcc A.o B.o C.o -o main
```

```
☐ gcc -c A.c -o A.o
☐ gcc A.o B.o C.o -o main
```

```
gcc -c A.c -o A.o
☐ gcc -c B.c -o B.o
☐ gcc -c C.c -o C.o
gcc A.o B.o C.o D.o -o main

gcc -c A.c -o A.o
☐ gcc -c B.c -o B.o
gcc A.o B.o C.o -o main
```

Domanda 2

Si supponga di avere questo frammento di codice C:

```
for (int i = 0; i < n; i++) {
    if (v[i] < 50) {
        if (v[i] < 25) {
            v[i] = 0;
        } else {
            v[i] = 1;
        }
    }
}
```

e si assuma di avere variabili v e n definite e del tipo corretto. La variabile v punta a un vettore di n elementi uniformemente distribuiti tra 0 e 100.

Quale delle seguenti è la versione branchless del codice precedente? La versione branchless migliorerebbe le prestazioni? Si assuma che il compilatore non ottimizzi.

☐ `int p = (v[i] < 50);`
`int q = (v[i] < 25);`
`v[i] = p*(1-q) + (1-p)*v[i];`
Non migliorerebbe le prestazioni

☐ `int p = (v[i] < 50) + (v[i] < 25);`
`v[i] += p*v[i];`
Non migliorerebbe le prestazioni

☐ `int p = (v[i] < 50);`
`int q = (v[i] < 25);`
`v[i] = p*(1-q) + (1-p)*v[i];`
Migliorerebbe le prestazioni

☐ `int p = (v[i] < 50) + (v[i] < 25);`
`v[i] += p*v[i];`
Migliorerebbe le prestazioni

Domanda 3

Date le tre seguenti strutture dati:

- A Un array di strutture ognuna con tre campi a , b e c ;
- B Una struttura di array con tre vettori per a , b e c ;
- C Una lista concatenata doppia in cui appaiono, in sequenza, tutti i valori di a , seguiti dai nodi con tutti i valori di b , seguiti dai nodi con tutti i valori di c .

Si assuma che si voglia calcolare la somma di tutti i valori del campo b . Quale struttura fornirebbe la migliore località di memoria?

☐ A

☐ B

☐ C

☐ Sarebbero equivalenti in quanto l'accesso è sequenziale in ogni caso

Domanda 4

Si supponga di avere del codice che fa uso delle intrinsics del compilatore per utilizzare le istruzioni vettoriali AVX2. Quale delle seguenti affermazioni è corretta?

☐ Per rendere il codice portabile su altre architetture è sufficiente compilare senza `-mavx`

☐ Il codice è portabile su altre architetture solo se utilizzano registri vettoriali di uguale lunghezza (256 bit in questo caso)

☐ Il codice sarà necessariamente più veloce del codice C standard equivalente in quanto il compilatore non è in grado di generare in modo autonomo istruzioni vettoriali

☐ Il codice è legato alla specifica architettura che implementa quelle estensioni vettoriali

Domanda 5

Si supponga di effettuare il mapping in memoria di un file di 1TB con `mmap` quando la memoria fisica disponibile è di 64GB. Quale delle seguenti affermazioni è corretta?

- | | |
|--|--|
| <p>☐ La chiamata a <code>mmap</code> fallisce in quanto deve caricare l'intero file in memoria</p> <p>☐ La chiamata a <code>mmap</code> ha successo ma al primo accesso alla memoria ritornata viene generato un <code>bus error</code> o un <code>segmentation fault</code></p> | <p>☐ La chiamata a <code>mmap</code> fallisce in quanto, anche se il file non viene caricato immediatamente in memoria, la sua dimensione deve essere minore della memoria fisica</p> <p>☐ La chiamata a <code>mmap</code> ha successo e sarà caricato in memoria solo il contenuto del file a cui accediamo</p> |
|--|--|
-
-
-

Domanda 6

Dato il seguente codice C facente uso di OpenMP:

```
int main(int argc, int * argv[])
{
    const int n = 1000;
    float * v1 = random_vector(n);
    float * v2 = random_vector(n);
    float * M = zero_matrix(n, n);
    #pragma omp /* TODO 1 */
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            M[i * n + j] = v1[i] * v2[j]
        }
    }
    float d = 1;
    #pragma omp /* TODO 2 */
    for (int i = 0; i < n; i++) {
        d *= M[i * n + i];
    }
    return 0;
}
```

Si supponga che le funzioni `random_vector` e `zero_matrix` siano correttamente definite. Al posto dei TODO che codice può essere inserito affinché il programma funzioni correttamente?

- | | |
|--|---|
| ☐ parallel nel primo e single nel secondo | ☐ parallel for nel primo e solamente parallel nel secondo |
| ☐ parallel for collapse(2) nel primo e parallel for nel secondo | ☐ parallel for collapse(2) nel primo e parallel for reduction(* : d) nel secondo |
-
-
-

Domanda 7

Dato il seguente codice Python:

```
class A:
```

```
    def __init__(self, x):
        self.x = x
```

```
    def __call__(self):
        print(self.x)
```

```
class B(A):
```

```
    def __call__(self, y):
        return A(y+2*self.x)
```

```
class C(B):
```

```
    def __call__(self):
        return B(3)(self.x)
```

```
C(2)()
```

cosa viene stampato a schermo?

☐ 7

☐ 8

☐ Il codice genera una eccezione

☐ <__main__.A object at 0x10a437620>

Domanda 8

Dato il seguente codice Python:

```
def f(g, x):  
    def h(y):  
        return g(x, y)  
    return h
```

```
func = f(lambda x, y: [x] + y + [x], 3)
```

```
print(func([2,4,6]))
```

Cosa viene stampato a schermo?

☐ [3, 2, 4, 6, 3]

☐ [8, 10, 12]

☐ [2, 4, 6, 3, 2, 4, 6]

☐ [5, 4, 9]

Domanda 9

Dato il seguente codice Python:

```
def deco(n, m):  
    def g(f):  
        def h(k):  
            return [f(k + [i]) for i in range(n, m)]  
        return h  
    return g
```

```
@deco(2, 4)
```

```
def k(lst):  
    return sum([x**2 for x in lst])
```

```
print(k([1,2,3]))
```

Quale è il valore stampato a schermo?

☐ 14

☐ [18, 23]

☐ 41

☐ [16, 17]

Domanda 10

Dato il seguente codice Python:

```
def f(lst, k):  
    for x in lst:  
        if x < k:  
            yield x
```

```
h = f([1,3,4,5,6,7,1,4,2], 4)  
lst = [x for x in h]  
print(lst)
```

Quali sono i valori stampati a schermo?

☐ [1, 3, None, None, None, None, 1, None, 2]

☐ [1, 3, 1, 2]

☐ [1, 3, 1, 2, 1, 3, 1, 2, ...]

☐ [[1], [3], [3, 5, 6, 7, 1], [4, 2]]

Parte 2

Esercizio 1

Si completi la seguente funzione C in cui M è una matrice $n \times n$ rappresentata in forma row-major e `sums` è un vettore di n elementi (inizialmente zero) in modo che:

- in ogni `sums[j]` sia contenuta una somma pesata dei valori della **j-esima colonna** di **M** in cui ogni valore è pesato con peso dato dall'indice di riga. Ovvero $sums[j] = \sum_{i=0}^{n-1} i \times M_{i,j}$ (righe e colonne indicizzate da zero).
- la funzione ritorna la somma dei valori in `sums[j]`.

Entrambe le operazioni devono essere correttamente parallelizzate con OpenMP.

```
float sum_of_col_wsum(float * M, float * sums, int n)
{
```

[illegible]

```
return sum;
}
```

Esercizio 2

Si scriva un decoratore che, dato un parametro `n` possa decorare una funzione `f` di un numero arbitrario di argomenti. La funzione decorata, quando chiamata con argomenti `x` deve ritornare una lista di `n` elementi: `[f(x), f(x), ..., f(x)]` prestando attenzione che la funzione `f` originale sia chiamata una volta sola per generare la lista:

```
def replicate(n):
    def g(f):
```

[illegible]

```

return g

```